📄 Plan du PDF : « Préparer l'environnement J2EE »

1️⃣ Introduction

- Qu'est-ce que J2EE / Jakarta EE
- Différence avec Java standard et PHP
- Objectif : configurer un environnement complet pour développer des applications web Java

2️⃣ Prérequis

- **Système** : Windows (ou Linux/Mac)
- **Java JDK** : version 11 ou 21
- **IDE** : Eclipse IDE for Enterprise Java Developers
- **Serveur** : Apache Tomcat 10.x
- **Base de données** : MySQL (ou MariaDB)
- **Outils optionnels** : navigateur web moderne, Postman

3️⃣ Étape 1 : Installer Java JDK

1. Télécharger JDK depuis Temurin / Oracle
2. Vérifier l'installation :

```
java -version
javac -version
```

3. Configurer la variable d'environnement **JAVA_HOME**
4. Ajouter `JAVA_HOME/bin` au **PATH**

4 Étape 2 : Installer Eclipse IDE

1. Télécharger **Eclipse IDE for Enterprise Java Developers**
2. Décompresser et lancer Eclipse
3. Configurer le workspace (par exemple `C:/workspace`)

5 Étape 3 : Installer Tomcat

1. Télécharger Tomcat 10.x
2. Décompresser dans `C:/apache-tomcat-10.1.52`
3. Tester le serveur en ouvrant `http://localhost:8080`

6 Étape 4 : Configurer Tomcat dans Eclipse

1. **Créer un nouveau serveur** :
   a. Servers → New → Server → Apache → Tomcat v10.1
   b. Sélectionner le dossier Tomcat téléchargé
2. **Changer Server Locations** si nécessaire :
   a. `Use Tomcat installation` pour utiliser le vrai dossier `webapps`
   b. Important : si bloqué, supprimer le serveur et le recréer
3. Ajouter un projet web au serveur → vérifier qu'il se déploie correctement

7 Étape 5 : Installer MySQL

1. Télécharger MySQL Community Server
2. Installer et créer un utilisateur
3. Tester la connexion avec **MySQL Workbench** ou via JDBC dans Java

8 Étape 6 : Créer un projet J2EE dans Eclipse

1. File → New → Dynamic Web Project
2. Configurer le projet :

   a. Target runtime → Tomcat v10.1
   b. Dynamic web module version → 4.0
3. Ajouter JSP / Servlet
4. Déployer sur le serveur → tester via navigateur

9 Étape 7 : Vérification finale

- Tester les URLs :
  - http://localhost:8080/ → ROOT
  - http://localhost:8080/MonProjet/ → projet déployé
- Vérifier la console Eclipse pour les erreurs

10 Astuces / Résolution de problèmes

- HTTP 404 → vérifier le nom du projet et les fichiers JSP
- ClassNotFound Bootstrap → vérifier Server Locations
- Erreurs JDK → vérifier JAVA_HOME et PATH
- Logs Tomcat → workspace/.metadata/.plugins/org.eclipse.wst.server.core/tmp0/logs

11 Conclusion

- Résumé de la configuration
- Conseils pour bien démarrer J2EE

Fichier index.jsp

```
<html>
<body>
<h1>Hello J2EE</h1>
</body>
</html>
```

- Inclure **arborescence complète du projet** pour clarifier le chemin `webapps/MonProjet`

📄 Erreurs rencontrées et solutions (Expérience Personnelle)

1️⃣ HTTP 404 – La ressource demandée [/app/] ou [/] n'est pas disponible

Symptôme :

- Ouvrir `http://localhost:8080/` ou `http://localhost:8080/app/` → page 404

Cause :

- Tomcat fonctionne, mais le dossier correspondant à l'URL n'existe pas dans `webapps`
- Aucun fichier d'accueil (`index.jsp` ou `index.html`) dans le dossier ROOT ou projet

Solution :

- Vérifier que le dossier existe : `webapps/ROOT` ou `webapps/app`
- Ajouter un fichier `index.jsp` ou `index.html`
- Ou configurer `WEB-INF/web.xml` avec `<welcome-file-list>`

2️⃣ Logs Tomcat vides / non trouvés

Symptôme :

- Le dossier `logs` de Tomcat est vide
- Impossible de voir les erreurs dans `catalina.out`

Cause :

- Eclipse utilise `Use workspace metadata` → Tomcat interne dans le workspace, pas le dossier téléchargé
- Les logs sont dans :

```
workspace/.metadata/.plugins/org.eclipse.wst.server.core/tmp0/logs
```

Solution :

- Vérifier le vrai chemin des logs
- Ou changer Server Locations pour `Use Tomcat installation`

3 Server Locations bloqué dans Eclipse

Symptôme :

- Impossible de changer "Server Locations" → reste sur `Use workspace metadata`

Cause :

- Le serveur Tomcat a déjà été créé → Eclipse verrouille la configuration

Solution :

1. Arrêter le serveur
2. Supprimer le serveur (`Delete → Also delete server configuration`)
3. Recréer le serveur → maintenant tu peux choisir `Use Tomcat installation`

4 HTTP 404 pour /ehhehe/ mais fonctionne pour /ehhehe/NewFile.jsp

Symptôme :

- http://localhost:8080/ehhehe/ → 404
- http://localhost:8080/ehhehe/NewFile.jsp → fonctionne

Cause :

- Aucun **welcome file** défini
- Tomcat cherche automatiquement `index.jsp` ou `index.html` pour `/ehhehe/`

Solution :

- Renommer `NewFile.jsp` en `index.jsp`

- Ou définir `<welcome-file>NewFile.jsp</welcome-file>` dans `WEB-INF/web.xml`

5 Classe principale Bootstrap non trouvée (ClassNotFoundException)

Symptôme :

- `Server Tomcat failed to start`
- Erreur : `java.lang.ClassNotFoundException: org.apache.catalina.startup.Bootstrap`

Cause :

- Tomcat mal configuré dans Eclipse
- Server Locations incorrect → Eclipse ne trouve pas les jars nécessaires

Solution :

- Reconfigurer Tomcat → `Use Tomcat installation`
- Vérifier que le dossier `bin` contient `bootstrap.jar`
- Redémarrer Eclipse et le serveur

6 Ports ou conflit de versions Java

Symptôme :

- Serveur ne démarre pas
- Messages d'erreur sur JRE / version incompatible

Cause :

- Eclipse utilise un JRE non compatible avec Tomcat
- Par exemple, Tomcat 10 requiert Java 11+

Solution :

- Vérifier `Window → Preferences → Java → Installed JREs`

- Assigner JDK compatible
- Vérifier `Project → Build Path → Execution Environment`

☑ Résumé global

Erreur	Cause principale	Solution
HTTP 404 / ou /app/	Dossier ou index manquant	Créer index.jsp ou web.xml welcome-file
Logs vides	Tomcat workspace metadata	Chercher dans `tmp0/logs` ou changer Server Locations
Server Locations bloqué	Serveur déjà créé	Supprimer serveur → recréer
HTTP 404 /ehhehe/	Pas de welcome file	Renommer NewFile.jsp en index.jsp ou modifier web.xml
Bootstrap ClassNotFound	Mauvaise config Tomcat	Server Locations = Use Tomcat installation
Problème JDK / ports	JRE incompatible	Configurer JDK compatible dans Eclipse